



Faculté des
Mathématiques et de
l'Informatique

Language Theory

Formal Grammars

F. Nouioua

Formal Grammars: Definition

● **Definition.** A formal grammar is a 4-uplets $G = (N, \Sigma, P, S)$ where:

- N is the set of **non terminal symbols**, called **non terminal alphabet**.
- Σ is the set of **terminal symbols**, called **terminal alphabet**, such that N and Σ are disjoint sets.
- P is a finite sub-set of :

$$(N \cup \Sigma)^* N (N \cup \Sigma)^* \times (N \cup \Sigma)^*$$

- an element (α, β) of P , denoted by $\alpha \rightarrow \beta$ is called a **production rule or rewriting rule**.

α is called the left part of the rule

β is called the right part of the rule

- S is an element of N called **the axiom** of the grammar.

Formal Grammars: Notation

For the sake of simplification, if we have n rules having the same left-hand part:

$$\alpha \rightarrow \beta_1, \alpha \rightarrow \beta_2, \dots, \alpha \rightarrow \beta_n$$

We denote them as follows:

$$\alpha \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

Formal Grammars : Proto-Words

- The **proto-words** of a grammar $G = (N, \Sigma, P, S)$ are words constructed on the alphabet $\Sigma \cup N$. We recursively define them as follows:
 - S is a proto-word of G
 - if $\alpha\beta\gamma$ is a proto-word of G and $\beta \rightarrow \delta \in P$ then $\alpha\delta\gamma$ is a proto-word of G .
 - A proto-word of G which contains no non terminal symbol is called **a word generated by G** .
 - The **language generated by G** , denoted $L(G)$ is the set of words generated by G .

Formal Grammars: Derivation

- The operation which consists in generating a proto-word $\alpha\delta\gamma$ from a proto-word $\alpha\beta\gamma$ and a *production* rule r of the form $\beta \rightarrow \delta$ is called the derivation operation. It is denoted by a double arrow:

$$\alpha\beta\gamma \Rightarrow \alpha\delta\gamma$$

- We denote by $\alpha \xRightarrow{k} \beta$ the fact that β is derived from α in k steps
- We also define the notations $\alpha \xRightarrow{+} \beta$ and $\alpha \xRightarrow{*} \beta$ as follows:
 - $\alpha \xRightarrow{+} \beta \equiv \alpha \xRightarrow{k} \beta$ with $k > 0$
 - $\alpha \xRightarrow{*} \beta \equiv \alpha \xRightarrow{k} \beta$ with $k \geq 0$

Formal Grammars: Generated Language

- $L(G)$ is defined as follows:

$$L(G) = \{w \in \Sigma^* \mid S \xRightarrow{+} w\}$$

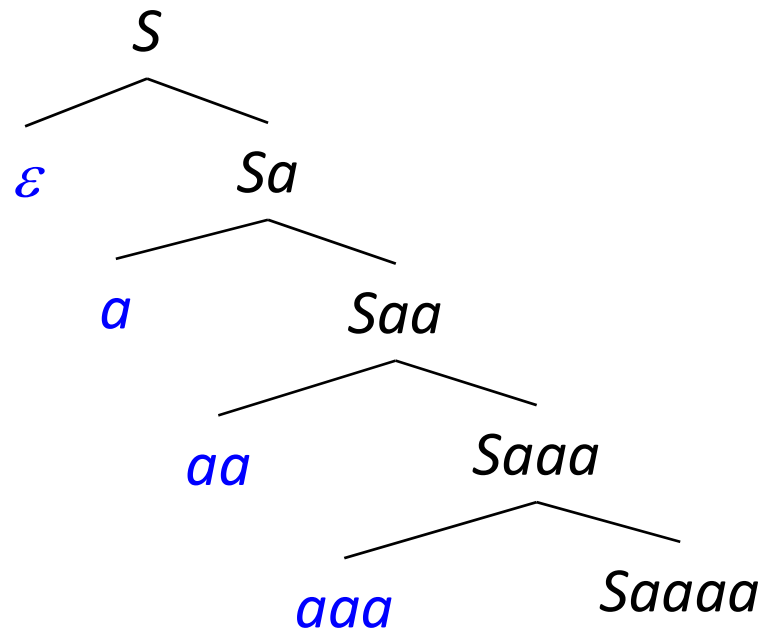
- Two grammars G and G' are equivalent if $L(G) = L(G')$.

Formal Grammars: Examples

$$L_1 = \{\varepsilon, a, aa, \dots\}$$

$$G = (\{S\}, \{a\}, \{S \rightarrow Sa \mid \varepsilon\}, S)$$

Sub-set of proto-words of G

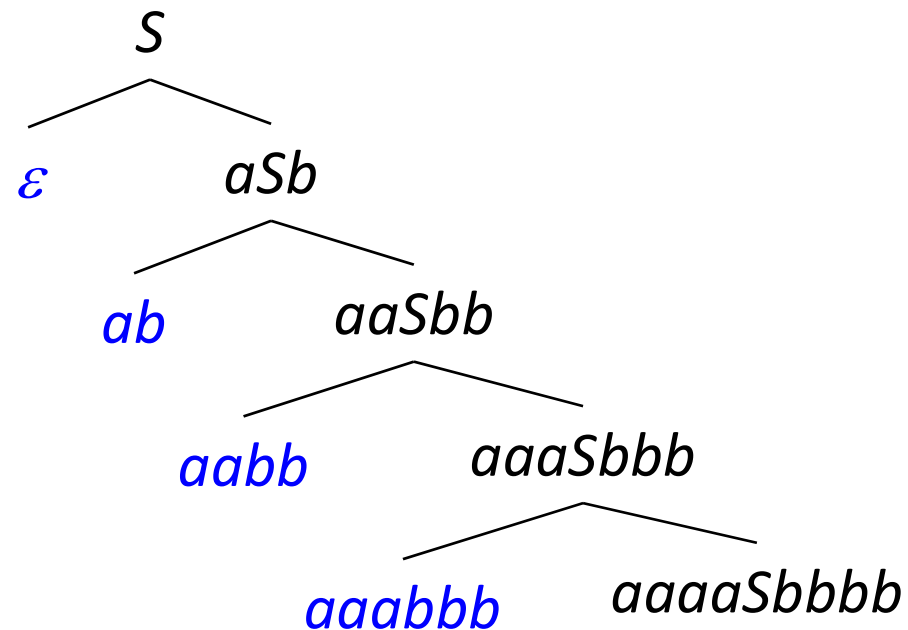


Formal Grammars: Examples

$$L_2 = \{\varepsilon, ab, aabb, aaabbb, \dots\}$$

$$G = (\{S\}, \{a, b\}, \{S \rightarrow aSb \mid \varepsilon\}, S)$$

Sub-set of proto-words of G

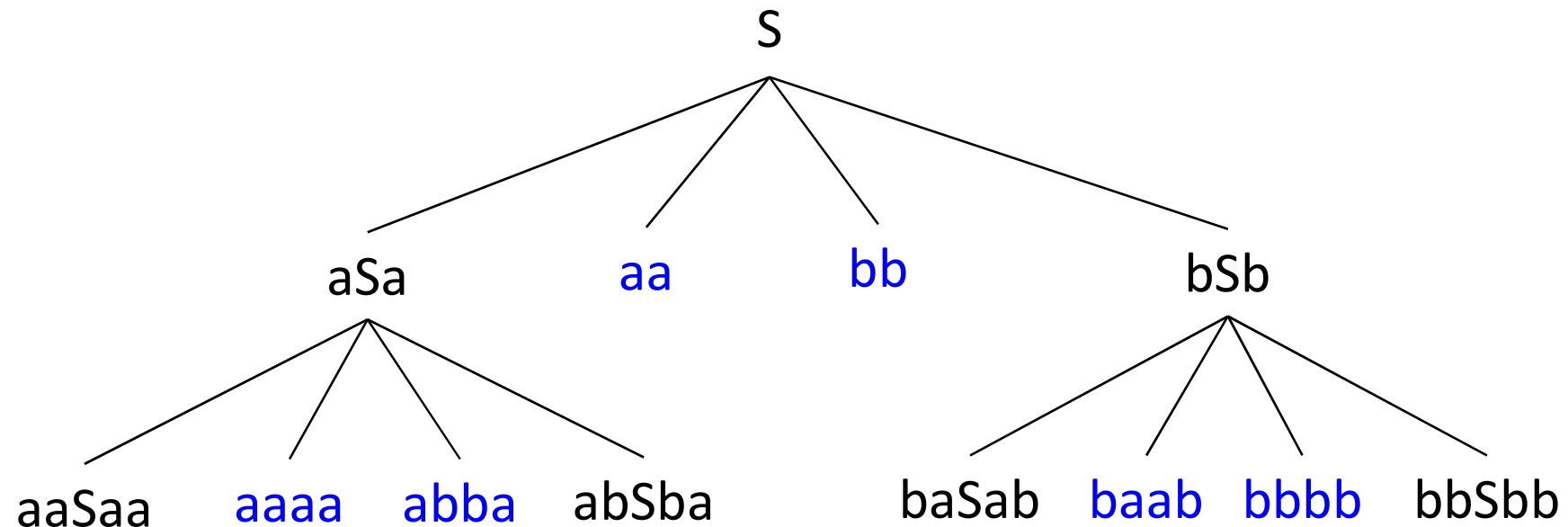


Formal Grammars: Examples

$$L_3 = \{\varepsilon, aa, bb, aaaa, abba, baab, bbbb, \dots\}$$

$$G = (\{S\}, \{a, b\}, \{S \rightarrow aSa \mid bSb \mid aa \mid bb\}, S)$$

Sub-set of proto-words of G

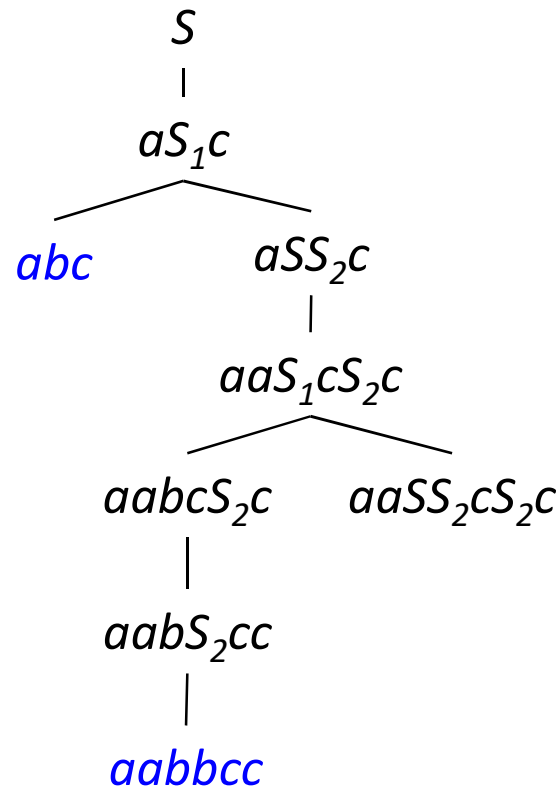


Formal Grammars: Examples

$$L_4 = \{\varepsilon, abc, aabbcc, aaabbbccc, \dots\}$$

$$G = (\{S, S_1, S_2\}, \{a, b, c\}, \{S \rightarrow aS_1b, S_1 \rightarrow b \mid SS_2, cS_2 \rightarrow S_2c, bS_2 \rightarrow bb\}, S)$$

Sub-set of proto-words of G



Formal Grammars: Derivation direction

$$G = (\{E, T, F\}, \{+, *, a\}, \{E \rightarrow T + E \mid T \rightarrow, T \rightarrow F * T \mid F, F \rightarrow a\}, E)$$

- The proto-words generated during a derivation can contain more than one non terminal symbol:

$$E \Rightarrow T + E \Rightarrow T + T \Rightarrow F + T \Rightarrow F + F * T \Rightarrow F + a * T \Rightarrow F + a * F \Rightarrow a + a * F \Rightarrow a + a * a$$

- **Right derivation:** we rewrite the rightmost non terminal:

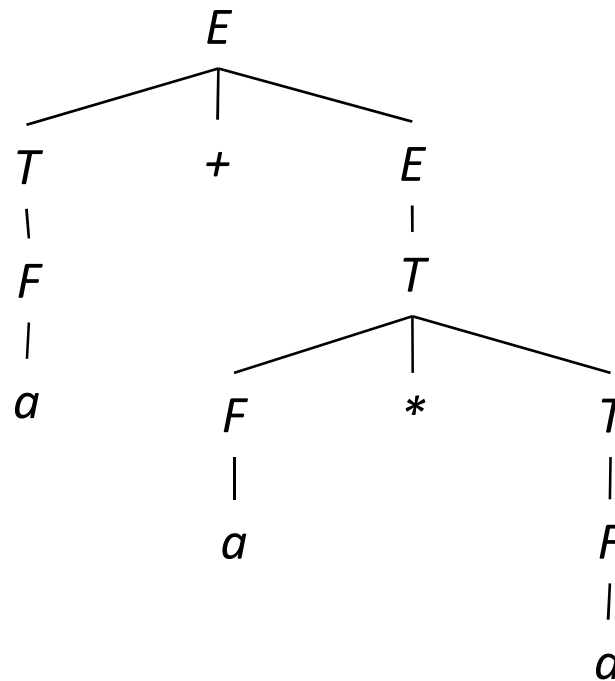
$$E \Rightarrow T + E \Rightarrow T + T \Rightarrow T + F * T \Rightarrow T + F * F \Rightarrow T + F * a \Rightarrow T + a * a \Rightarrow F + a * a \Rightarrow a + a * a$$

- **Left derivation:** we rewrite the leftmost non terminal:

$$E \Rightarrow T + E \Rightarrow F + E \Rightarrow a + E \Rightarrow a + T \Rightarrow a + F * T \Rightarrow a + a * T \Rightarrow a + a * F \Rightarrow a + a * a$$

Formal Grammars: Derivation tree

A derivation tree for $G = (N, \Sigma, P, S)$ is an ordered and a labeled tree where labels belong to the set $N \cup \Sigma \cup \{\varepsilon\}$. If a node of the tree is labeled by the non terminal A and its children labeled $X_1, X_2, \dots, X_n$ then the rule $A \rightarrow X_1, X_2, \dots, X_n$ belongs to P .



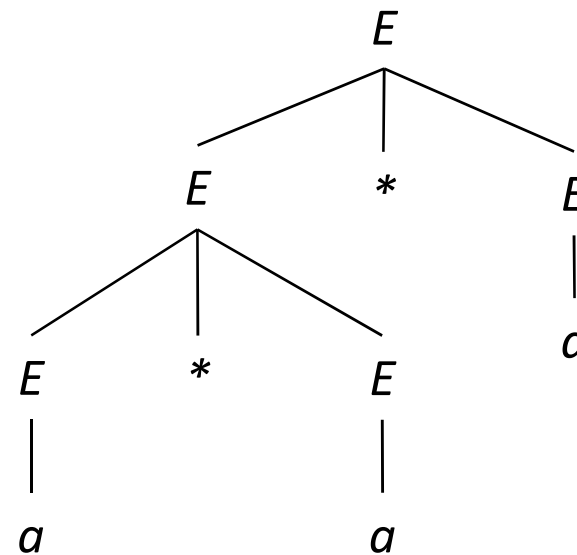
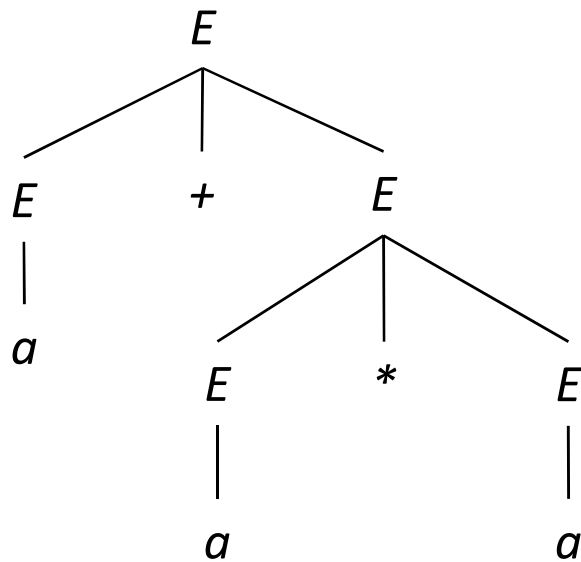
Formal Grammars: Derivation tree

- A derivation tree indicates the rules which have been used in a derivation, but not in the order in which they have been used.
- To a derivation tree corresponds **only one** rightmost derivation and **only one** leftmost derivation.

Formal Grammars: Ambiguity

- A grammar G is **ambiguous** if there exist at least a word m in $L(G)$ having more than one derivation tree.
- **Example.** $E \rightarrow E + E \mid E * E \mid a$

$$w = a + a * a$$



Types of rules

Grammars are classified depending on the form of their production rules. We define five types of production rules:

Types of rules

Grammars are classified depending on the form of their production rules. We define five types of production rules:

- A rule is **right-regular** if and only if it is of the form $A \rightarrow xB$ or $A \rightarrow x$ with $A, B \in N$ and $x \in \Sigma$.

Types of rules

Grammars are classified depending on the form of their production rules. We define five types of production rules:

- A rule is **right-regular** if and only if it is of the form $A \rightarrow xB$ or $A \rightarrow x$ with $A, B \in N$ and $x \in \Sigma$.
- A rule is **left-regular** if and only if it is of the form $A \rightarrow Bx$ or $A \rightarrow x$ with $A, B \in N$ and $x \in \Sigma$.

Types of rules

Grammars are classified depending on the form of their production rules. We define five types of production rules:

- A rule is **right-regular** if and only if it is of the form $A \rightarrow xB$ or $A \rightarrow x$ with $A, B \in N$ and $x \in \Sigma$.
- A rule is **left-regular** if and only if it is of the form $A \rightarrow Bx$ ou $A \rightarrow x$ with $A, B \in N$ and $x \in \Sigma$.
- A rule $A \rightarrow \alpha$ is **context-free** if and only if: $A \in N$ and $\alpha \in (N \cup \Sigma)^*$

Types of rules

Grammars are classified depending on the form of their production rules. We define five types of production rules:

- A rule is **right-regular** if and only if it is of the form $A \rightarrow xB$ or $A \rightarrow x$ with $A, B \in N$ and $x \in \Sigma$.
- A rule is **left-regular** if and only if it is of the form $A \rightarrow Bx$ or $A \rightarrow x$ with $A, B \in N$ and $x \in \Sigma$.
- A rule $A \rightarrow \alpha$ is **context-free** if and only if: $A \in N$ and $\alpha \in (N \cup \Sigma)^*$
- A rule $\alpha \rightarrow \beta$ is a **contextual rule** if and only if: $\alpha = gAd$ and $\beta = gBd$ with, $d, B \in (N \cup \Sigma)^*$ and $A \in N$.

The name “contextual” comes from the fact that A can be rewritten as B only in the context $g-d$.

Types of rules

Grammars are classified depending on the form of their production rules. We define five types of production rules:

- A rule is **right-regular** if and only if it is of the form $A \rightarrow xB$ or $A \rightarrow x$ with $A, B \in N$ and $x \in \Sigma$.
- A rule is **left-regular** if and only if it is of the form $A \rightarrow Bx$ or $A \rightarrow x$ with $A, B \in N$ and $x \in \Sigma$.
- A rule $A \rightarrow \alpha$ is **context-free** if and only if: $A \in N$ and $\alpha \in (N \cup \Sigma)^*$
- A rule $\alpha \rightarrow \beta$ is a **contextual rule** if and only if: $\alpha = gAd$ and $\beta = gBd$ with, $d, B \in (N \cup \Sigma)^*$ and $A \in N$.

The name “contextual” comes from the fact that A can be rewritten as B only in the context $g-d$.

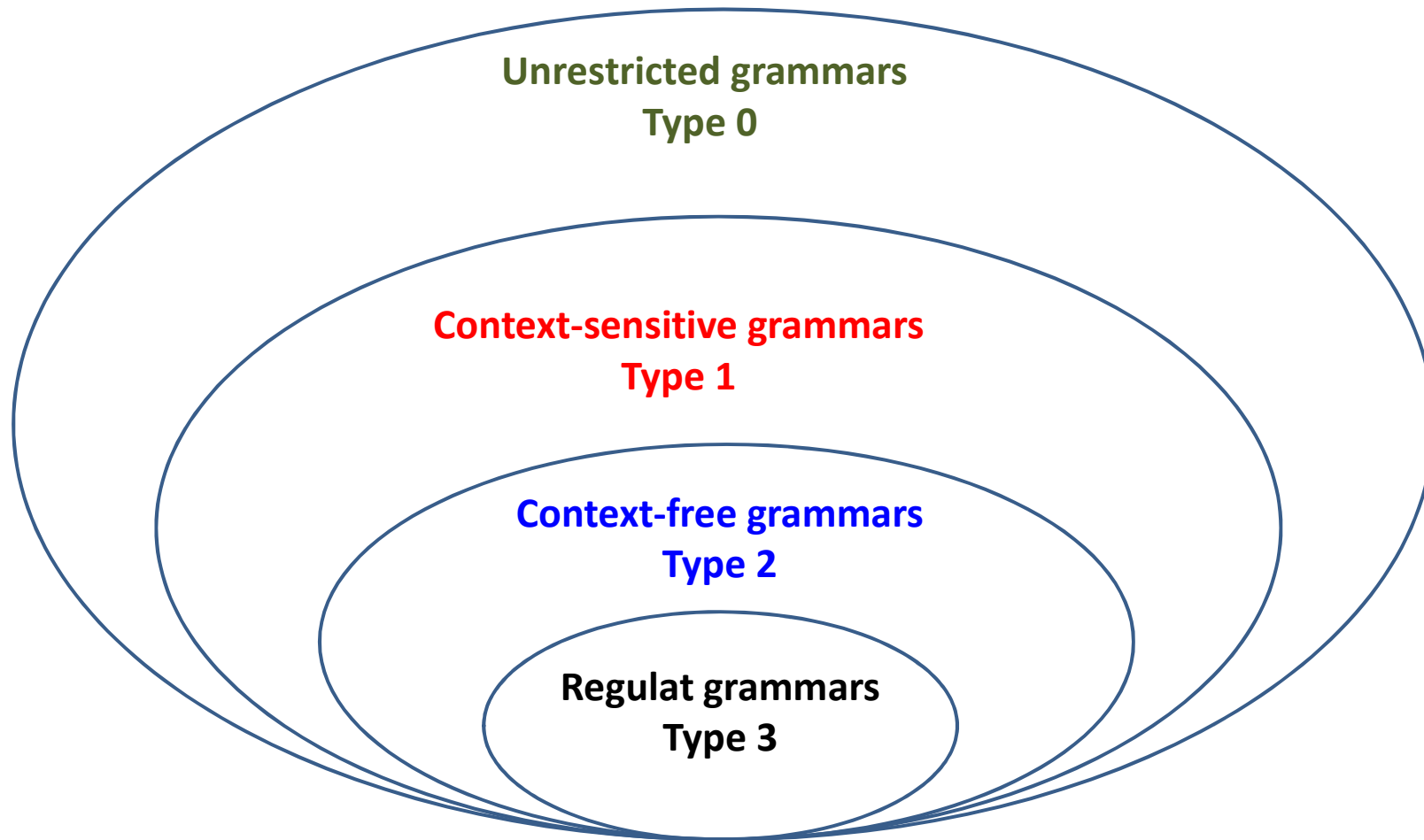
- A rule $\alpha \rightarrow \beta$ is **an unrestricted rule** if and only if: $|\alpha| \geq 1$

Types of grammars

A grammar is:

- **regular or of type 3** if it is right-regular or left-regular. A grammar is **left-regular** if **all** its rules are left-regular. A grammar is **right-regular** if **all** its rules are right-regular;
- **context-free or of type 2** if **all** its production rules are context-free.
- **context-sensitive or of type 1** if **all** its production rules are contextual.
- **Unrestricted or of type 0** if **all** its production rules are unrestricted.

Chomsky hierarchy



Types of languages

A language which can be generated by a grammar of type k but not by a grammar of type k' with $k' > k$, is called **language of type k** .

Type	Nom
3	Regulier or Rational
2	Context-free
1	Contexte-sensitive
0	Recursively enumerable

Examples: Rational languages

- $L = \{w \in \{a, b\}^*\}$

Examples: Rational languages

- $L = \{w \in \{a, b\}^*\}$

$$G = (\{S\}, \{a, b\}, \{S \rightarrow aS \mid bS \mid \varepsilon\}, S)$$

Examples: Rational languages

- $L = \{w \in \{a, b\}^*\}$

$$G = (\{S\}, \{a, b\}, \{S \rightarrow aS \mid bS \mid \varepsilon\}, S)$$

- $L = \{w \in \{a, b\}^* \mid |w|_a \bmod 2 = 0\}$

Examples: Rational languages

- $L = \{w \in \{a, b\}^*\}$

$$G = (\{S\}, \{a, b\}, \{S \rightarrow aS \mid bS \mid \varepsilon\}, S)$$

- $L = \{w \in \{a, b\}^* \mid |w|_a \bmod 2 = 0\}$

$$G = (\{S, T\}, \{a, b\}, \{S \rightarrow aT \mid bS \mid \varepsilon, T \rightarrow aS \mid bT\}, S)$$

Examples: Rational languages

- $L = \{w \in \{a, b\}^*\}$

$$G = (\{S\}, \{a, b\}, \{S \rightarrow aS \mid bS \mid \varepsilon\}, S)$$

- $L = \{w \in \{a, b\}^* \mid |w|_a \bmod 2 = 0\}$

$$G = (\{S, T\}, \{a, b\}, \{S \rightarrow aT \mid bS \mid \varepsilon, T \rightarrow aS \mid bT\}, S)$$

- $L = \{w \in \{a, b\}^* \mid w = xaaa \text{ with } x \in \{a, b\}^*\}$

Examples: Rational languages

- $L = \{w \in \{a, b\}^*\}$

$$G = (\{S\}, \{a, b\}, \{S \rightarrow aS \mid bS \mid \varepsilon\}, S)$$

- $L = \{w \in \{a, b\}^* \mid |w|_a \bmod 2 = 0\}$

$$G = (\{S, T\}, \{a, b\}, \{S \rightarrow aT \mid bS \mid \varepsilon, T \rightarrow aS \mid bT\}, S)$$

- $L = \{w \in \{a, b\}^* \mid w = xaaa \text{ with } x \in \{a, b\}^*\}$

$$G = (\{S, T, U\}, \{a, b\}, \{S \rightarrow aS \mid bS \mid aT, T \rightarrow aU, U \rightarrow a\}, S)$$

Examples: Rational languages

- $L = \{w \in \{a, b\}^*\}$

$$G = (\{S\}, \{a, b\}, \{S \rightarrow aS \mid bS \mid \varepsilon\}, S)$$

- $L = \{w \in \{a, b\}^* \mid |w|_a \bmod 2 = 0\}$

$$G = (\{S, T\}, \{a, b\}, \{S \rightarrow aT \mid bS \mid \varepsilon, T \rightarrow aS \mid bT\}, S)$$

- $L = \{w \in \{a, b\}^* \mid w = xaaa \text{ with } x \in \{a, b\}^*\}$

$$G = (\{S, T, U\}, \{a, b\}, \{S \rightarrow aS \mid bS \mid aT, T \rightarrow aU, U \rightarrow a\}, S)$$

- $L = \{w \in \{a, b\}^* \mid |w|_a \bmod 2 = 0 \text{ and } |w|_b \bmod 2 = 0\}$

Examples: Rational languages

- $L = \{w \in \{a, b\}^*\}$

$$G = (\{S\}, \{a, b\}, \{S \rightarrow aS \mid bS \mid \varepsilon\}, S)$$

- $L = \{w \in \{a, b\}^* \mid |w|_a \bmod 2 = 0\}$

$$G = (\{S, T\}, \{a, b\}, \{S \rightarrow aT \mid bS \mid \varepsilon, T \rightarrow aS \mid bT\}, S)$$

- $L = \{w \in \{a, b\}^* \mid w = xaaa \text{ with } x \in \{a, b\}^*\}$

$$G = (\{S, T, U\}, \{a, b\}, \{S \rightarrow aS \mid bS \mid aT, T \rightarrow aU, U \rightarrow a\}, S)$$

- $L = \{w \in \{a, b\}^* \mid |w|_a \bmod 2 = 0 \text{ and } |w|_b \bmod 2 = 0\}$

$$G = (\{S, T, U, V\}, \{a, b\},$$

$$\{S \rightarrow aT \mid bU, T \rightarrow aS \mid bV, V \rightarrow aU \mid bT, U \rightarrow aV \mid bS \mid \varepsilon\}, S)$$

Examples: Context-free languages

- $L = \{a^n b^n \mid n \geq 0\}$

Examples: Context-free languages

- $L = \{a^n b^n \mid n \geq 0\}$

$$G = (\{S\}, \{a, b\}, \{S \rightarrow aSb \mid \varepsilon\}, S)$$

Examples: Context-free languages

- $L = \{a^n b^n \mid n \geq 0\}$

$$G = (\{S\}, \{a, b\}, \{S \rightarrow aSb \mid \varepsilon\}, S)$$

- $L = \{ww^{-1} \mid w \in \{a, b\}^*\}$ (*mirror language*)

Examples: Context-free languages

- $L = \{a^n b^n \mid n \geq 0\}$

$$G = (\{S\}, \{a, b\}, \{S \rightarrow aSb \mid \varepsilon\}, S)$$

- $L = \{ww^{-1} \mid w \in \{a, b\}^*\}$ (*mirror language*)

$$G = (\{S, T\}, \{a, b\}, \{S \rightarrow aSa \mid bSb \mid aa \mid bb\}, S)$$

Examples : Context-sensitive languages

- $L = \{a^n b^n c^n \mid n \geq 0\}$

$$G = (\{S, B, W, X\}, \{a, b, c\}, \{ S \rightarrow abc \mid aSBc,$$

$$cB \rightarrow WB$$

$$WB \rightarrow WX$$

$$WX \rightarrow BX$$

$$BX \rightarrow BC$$

$$bB \rightarrow bb\},$$

S)

Examples : Context-sensitive languages

Derivation of $a^3b^3c^3$

$S \Rightarrow_2 aSBc$
 $\Rightarrow_2 aaSBcBc$
 $\Rightarrow_1 aaabcBcBc$
 $\Rightarrow_3 aaabWBcBc$
 $\Rightarrow_4 aaabWXcBc$
 $\Rightarrow_5 aaabBXcBc$
 $\Rightarrow_6 aaabBccBc$
 $\Rightarrow_3 aaabBcWBc$
 $\Rightarrow_4 aaabBcWXc$
 $\Rightarrow_5 aaabBcBXc$
 $\Rightarrow_6 aaabBcBcc$
 $\Rightarrow_3 aaabBWBcc$
 $\Rightarrow_4 aaabBWXcc$
 $\Rightarrow_5 aaabBBXcc$
 $\Rightarrow_6 aaabBBccc$
 $\Rightarrow_7 aaabbBccc$
 $\Rightarrow_7 aaabbbccc$

Examples : recursively enumerable languages

- $L = \{w\#w \mid w \in \{a, b\}^*\}$
- $L = \{a^{2^n} \mid n \geq 0\}$
- $L = \{\#x_1\#x_2\# \dots \#x_n \mid x_i \in \{0, 1\}^* \text{ and } x_i \neq x_j \text{ for all } i \neq j\}$

Grammar vs Recognizer

- A **grammar** of a language L generates all the words belonging to L .
- A **recognizer** for a language L is an **abstract machine** which takes as input a word and answers **yes** if w belongs to L and **no** otherwise.
- For each type of grammar, there exists a type of recognizers which defines the same class of languages.

Grammar vs Recognizer

- A **grammar** of a language L generates all the words belonging to L .
- A **recognizer** for a language L is an **abstract machine** which takes as input a word and answers **yes** if w belongs to L and **no** otherwise.
- For each type of grammar, there exists a type of recognizers which defines the same class of languages.

Type of grammar	Type of recognizer
Regular	Finite State Automata
Context-free	Pushdown Automata
Context-sensitive	Linear Bounded Turing Machine
Unrestricted	Turing Machine

Equivalence between regular grammars and FSA

Every language generated by a regular grammar is recognizable.

- Given a **right-regular** $G = (N, \Sigma, P, S)$. The automaton with ε -transition $A = (Q, \Sigma, \delta, S, \{\varepsilon\})$ with:

- Q contains S and **all the suffixes of right-hand parts** of the rules of P .

- δ is defined as follows:

(1) If $A \in N$, then $\delta(A, \varepsilon) = \{\alpha \mid (A \rightarrow \alpha) \in P\}$;

(2) If $c \in T$ and $\alpha \in T^* \cup T^*.N$, then $\delta(c.\alpha, c) = \{\alpha\}$.

recognizes the language generated by G , i.e., $L(G) = L(A)$

Equivalence between regular grammars and FSA

For a left-regular grammar, it suffices:

- (1) to reverse the order of symbols in the right-hand sides of the rules to obtain a right-regular grammar which generates the same language but with all words reversed.
- (2) to construct the corresponding FSA by the method described above.
- (3) to reverse the direction of each edge in the obtained FSA and exchange the roles of initial and final states in order to obtain an automaton which recognizes the original language.

Equivalence between regular grammars and FSA

Every recognizable language is generated by a regular grammar

Consider the DFSA $A = (Q, \Sigma, \delta, q_0, F)$.

- If $q_0 \notin F$ (q_0 is not a final state), then we construct the equivalent grammar $G = (Q, \Sigma, P, q_0)$ where P is defined as follows:
 - If $\delta(q, c) = q'$, then put in P the rule : $q \rightarrow c.q'$.
 - Moreover, if $\delta(q, c) \in F$, also put in P the rule : $q \rightarrow c$.

Equivalence between regular grammars and FSA

Every recognizable language is generated by a regular grammar

Consider the DFSA $A = (Q, \Sigma, \delta, q_0, F)$.

- If $q_0 \notin F$ (q_0 is not a final state), then we construct the equivalent grammar $G = (Q, \Sigma, P, q_0)$ where P is defined as follows:
 - If $\delta(q, c) = q'$, then put in P the rule : $q \rightarrow c.q'$.
 - Moreover, if $\delta(q, c) \in F$, also put in P the rule : $q \rightarrow c$.
- If $q_0 \in F$ (q_0 is a final state)
 - Add to non terminals a new axiom S (in the place of q_0)
 - Add to P the rules : $S \rightarrow q_0 \mid \varepsilon$